

A Novel Hybrid Approach for Fault-Tolerant Control of UAVs based on Robust Reinforcement Learning

Yves Sohège¹, Marcos Quiñones-Grueiro² and Gregory Provan¹

Abstract—The control of complex autonomous systems has significantly improved in recent years and unmanned aerial vehicles (UAVs) have become popular in the research community. Although the use of UAVs is increasing, much work remains to guarantee fault-tolerant control (FTC) properties of these vehicles. Model-based controllers are the standard way to control UAVs, however obtaining models of the system and environment for every possible operating condition a UAV can experience in a real-world scenario is not feasible. Reinforcement Learning has shown promise in controlling complex systems but requires training in a simulator (requiring a model) of the system. Further, stability guarantees do not exist for learning-based controllers, which limits their large scale application in the real-world. We propose a novel hybrid FTC approach that uses a learned supervisory controller (together with low-level PID controllers) with key stability guarantees. We use a robust reinforcement learning approach to learn the supervisory control parameters and prove stability. We empirically validate our framework using trajectory-following experiments (in simulation) for a quadcopter subject to rotor faults, wind disturbances, and severe position and attitude noise.

I. INTRODUCTION

Researchers have developed fault-tolerant control strategies for UAVs by using model-based, learning-based, or combined techniques [1]. The performance and robustness of model-based FTC methods are highly dependent on the accuracy and comprehensiveness of the dynamic model of the system. A sophisticated model leads to a more complex control strategy, which compromises the tractability of the controller for real-time implementation. Learning-based FTC methods on the other hand allow developing control strategies by using data obtained from the system. Among learning-based FTC approaches, reinforcement learning (RL) has recently shown promising results [2], [3]. However, RL methods, e.g., [4], along with many other approaches, e.g., [5], lack control stability guarantees [1], which is of primary importance for the control of UAVs.

In this article, we propose a novel hybrid control strategy for FTC of UAVs on the task of maintaining the aircraft within a specified operation volume during flight. Our approach is based on using a hierarchical control architecture formed by a collection of model-based low-level controllers and a supervisory robust RL-based controller. We extend our prior work on FTC for quadcopters [6], [7] by applying robust RL methods, and validating our approach not just on

simple linear trajectories but on maintaining flight within a volume enclosing a linear trajectory.

The proposed approach integrates key properties of model-based and learning-based methods, as follows:

- 1) We propose a hybrid control strategy with stability guarantees under all but unrecoverable fault conditions [6], which maintains a satisfactory performance for novel faults under the assumption that the effect of a novel fault on the task to be solved is similar to previously experienced faults.
- 2) We leverage robust RL approaches, and use domain randomization to train the high-level controller.
- 3) We propose a novel FTC approach for UAV trajectory following tasks based on optimal operational volumes during flight.

We demonstrate stability guarantees for the proposed hybrid control approach and empirically validate it in simulations of flying in safe spaces of urban scenarios with a quadcopter testbed.

II. RELATED WORK

Commercially available UAVs present impressive trajectory tracking capabilities and typically utilize cascading control architectures that divide the task into position and attitude control [8], [9], [10]. Position controllers generate the required attitude reference and attitude controllers generate the required motor commands to follow a reference trajectory. The dynamic model of UAVs is well understood and a variety of controllers such as LQR, PID, and MPC, for both position and attitude exist [10], [11], [12]. Robustness is generally defined as the ability of a system to handle parameter variations within a bounded region, such as the varying weight of a UAV depending on a payload. In contrast, fault-tolerant control focuses on situations where system components fail, such as a rotor fault in a UAV. With advances in computational power, learning-based control approaches have made significant progress in UAVs control [13], [14], [15]. We next discuss the advantages and drawbacks of model-based and learning-based techniques for FTC of UAVs.

Model-based control is the most common way to control UAVs in the real-world. Using cascading LQR controllers, Mueller et al. [9] show that control of a quadcopter can be maintained even after the loss of one, two, or three rotors, highlighting the power of model-based design. By adding redundant controllers designed around different operating conditions the control system can switch between controllers when a disturbance is identified, which is referred to as

*This work was supported by:

¹Insight Centre for Data Analytics, University College Cork, Ireland yves.sohège@insight-centre.org
g.provan@cs.ucc.ie

²ISIS, Vanderbilt University, Nashville, Tennessee, USA
marcos.quinones@vanderbilt.edu

Multiple Model Adaptive Control [16]. However, obtaining models of the system and environment for every possible operating condition a UAV can experience in a real-world scenario is not feasible [6]. This is a major drawback of using pure model-based control methods.

Learning-based control, and specifically reinforcement learning, focus on learning the direct control mapping of measurements to motor commands [14], [15], [3]. Reinforcement learning trains policies requiring a vast amount of interactions with the system. Since real-world data is usually expensive to acquire, learning in simulation has become a popular training strategy. However, agents trained with a simulator tend to fail to generalize when transferred to the real environment due to a mismatch between the simulation and reality, usually called the *reality gap* or *sim-to-real* transfer problem [17], [18], [19]. The parameters of a simulator can be different from the real-world setting because of several reasons, e.g., the modeling errors, changes in the parameters over time, and adversarial disturbances. Robust reinforcement learning addresses the problem of finding an optimal policy that is robust against parameter uncertainties. Robustness to changing environmental dynamics is an important topic in reinforcement learning, which is crucial for overcoming the gap between simulation and reality. Agents tend to over-fit the domain on which they are trained because of an overestimation of the policy's performance given by a simulation optimization bias.

Hybrid control methods have become popular and increasingly successful for robust quadcopter control since both learning-based and model-based control methods have clear advantages and disadvantages. A typical approach is to use all prior knowledge, in terms of models available, to design robust system controllers and then use a learning component to compensate for unmodeled details. In [5], a hybrid control framework is used to make a quadcopter robust to cyber-physical attacks. A low-level cascading PID architecture is used for trajectory tracking and a neural network is used to compensate for the attacks on the system. Other hybrid approaches learn either inner or outer control loop only and use model-based controllers for the remainder [3], [20]. While this simplifies the complexity of the task, the *direct combination* of a model-based controller output with a learning-based controller output breaks the existing stability guarantees that make model-based controllers safe. To the best of the authors' knowledge, most hybrid approaches for UAV control suffer from this problem.

III. PRELIMINARIES

This section presents the background on RL approaches developed to obtain robust policies.

A. Robust Markov Decision Processes

A robust Markov decision process (R-MDP) is defined using a tuple $(S, A, R, \mathcal{P}, \gamma)$, where S is the state space, A the action space, $R : S \times A \rightarrow \mathbb{R}$ represents a reward function, $\gamma \in [0, 1]$ is called a discount factor, and $\mathcal{P}(s, a) \in \mathcal{M}(S)$ is an uncertainty set where $\mathcal{M}(S)$ is the set of

probability measures over next states $s' \in S$. An R-MDP defines how the next state of the system is determined by a conditional measure $p(s'|s, a) \in \mathcal{P}(s, a)$ where s is the current state and a is the action selected by an agent.

The robust RL literature usually adopts the assumption that $\mathcal{P}$ is structured as a cartesian product $\otimes_{s \in S, a \in A} \mathcal{P}_{s, a}$, which is also known as the state-action rectangularity assumption [21]. In RMDPs, this implies that nature can choose the worst-transition independently for each state and action.

In the standard MDP framework, an agent is defined by a policy $\pi : S \rightarrow p(A)$ that maps states to distributions over actions with the goal of maximizing the sum of the discounted rewards it receives over the future. The optimization criterion is the following

$$\mathcal{J}(\pi^*) = \max_{\pi \in \mathcal{O}} \mathcal{V}^\pi(f), \forall f \in S. \quad (1)$$

where the value function, $V^\pi : S \rightarrow \mathbb{R}$, defines the value of being in any given state s

$$V^\pi(s) = E \left[\sum_{t=0}^{\infty} \gamma^t R(s, a) \right], \forall s \in S. \quad (2)$$

Traditional RL algorithms require that the system dynamics and reward function do not change over time to be able to find an optimal deterministic Markovian policy satisfying 1. This property is clearly not satisfied in the R-MDP case. Therefore, different approaches have been proposed to tackle the uncertainty around the system dynamics. We next present two possible solutions.

B. Optimizing for the worst-case performance

One approach to incorporate the notion of robustness in standard RL algorithms is by transforming the optimization problem of equation 1 to account for the variability in the environment. An optimal robust value function V^* defined over the set of possible policies and possible changes in the system dynamics is defined as follows

$$\mathcal{J}_{\mathcal{R}}^{\square}(\pi^*) = \sup_{\pi} \inf_{\mathcal{P}} V_{\pi, p}(s) \quad (3)$$

The policy obtained by solving this optimization problem is robust in the sense of optimizing for the worst-case expected return considering the possible evolution of the environment. Mankowitz et al. [22] follow this approach by optimizing the worst-case squared temporal difference error. Abdullah et al. [23] propose to make problem 3 tractable by constraining the reachable states of the system according to the average Wasserstein ball around a reference dynamics.

C. Optimizing for the average-case performance

Robustness can be achieved by modifying the optimization problem 1 to maximize the expected return across the distribution of environments. The optimization criterion is formulated as follows [24]

$$\mathcal{J}_{\mathcal{R}}^{\perp}(\pi^*) = \sup_{\pi} \mathbb{E}_{p \in \mathcal{P}} V_{\pi, p}(s) \quad (4)$$

Domain randomization (DR) approaches; e.g., Tremblay et al. [25]) attempt to solve this optimization problem by

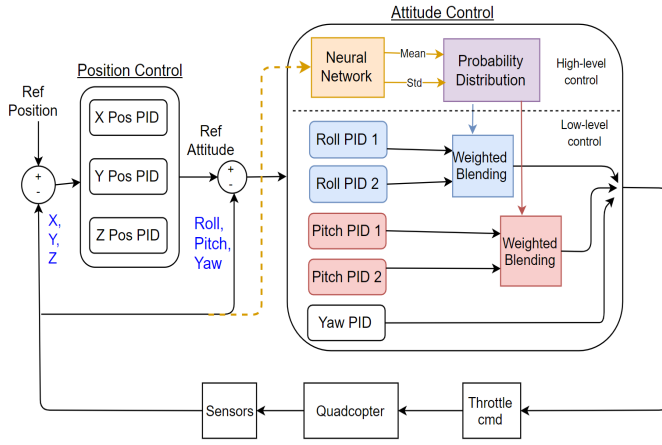


Fig. 1: Hybrid control architecture for fault-tolerant quadcopter trajectory tracking. Low-level control is established using cascading PID control with additional roll and pitch controllers. A supervisory neural network controller estimates the probability distribution used to sample the blending weights.

exposing the policy to be learned to a multitude of environments within the distribution of $\mathcal{P}$ in offline settings. Active Domain Randomization attempts to improve the generalization power of the agent by focusing the learning process on the parameter space which makes the dynamics of the environment difficult to control [26].

IV. HYBRID APPROACH FOR FTC

The approaches to solving the R-MDP problem presented in the previous section have some limitations if applied for FTC of UAVs. Optimizing for the worst-case performance leads to conservative and even pessimistic agents with sub-optimal policies. Optimizing for the average-case performance, on the other hand, makes the task too complex for the agent, thus resulting in low sample efficiency, which makes the training process a labor-intensive task leading sometimes to no convergence at all. In both cases, the resulting agent may perform optimally across all versions of the environment but will perform sub-optimally for each independent version. Moreover, in none of the above cases, the resulting agent provides stability guarantees.

We propose a hybrid approach that provides stability guarantees: we use a hybrid control architecture as shown in Figure 1. Through this control architecture, the agent learns to adapt to the changes in the environment by sampling the control actions from a set of model-based controllers. We now describe, in turn, the model-based controllers (low-level), and learning-based controller (high-level).

A. Model-based controllers

A cascading PID controller architecture, consisting of three position and three attitude controllers, is commonly used for trajectory tracking [10]. Fault tolerance is usually provided through additional PID controllers tuned for specific faults or disturbance conditions. We focus on tuning

roll and pitch controllers for different conditions. A set of controllers are tuned with **low-gains** which increases robustness to high attitude sensor noise. A different set is then tuned with **high-gains** to increase robustness to rotor faults and allow for aggressive reactions. Under nominal operating conditions, both controllers must be able to successfully follow a given trajectory.

We show trajectories executed by each set of controllers under the three different operating conditions in Figure 2. The leftmost plot shows nominal operating conditions. The green area highlights the operating region the quadcopter should stay inside to avoid collision with obstacles in the environment. The middle plot shows the same trajectory under heavy attitude noise (Gaussian noise with zero mean and 0.6 rad standard deviation). The low-gain controller is able to maintain control and complete the trajectory while the high-gain controller fails due to its aggressive nature. The rightmost plot highlights the opposite for rotor faults scenarios (20 % loss of power) where the aggressive reactions of the high-gain controllers are beneficial and the low-gain controllers are unable to maintain the quadcopter within the safe region. These contrasting types of required reactions are common across different disturbances and a single controller can generally not provide both quick reaction and low overshoot [27].

B. Learning-based controller

The proposed learning-based approach is based on the idea of sampling over the action space generated from a set of model-based controllers. The low-level controllers should be defined to accommodate the plausible variability in the distribution of environment dynamics ($\mathcal{P}$). In the case of FTC for UAVs, this variability is defined in terms of the fault scenarios to be considered as described in the previous section.

Once the set of low-level controllers is defined, the role of the supervisory agent is to map the low-level controller outputs into the final action space. We define the final action as taking a weighted sum of the low-level controller outputs. In other words, given a set of low-level controller outputs $a_1, a_2, \dots, a_m$, the final control output is $\mathbf{a} = \sum_{i=1}^m \varphi_{(i)} \cdot a_i$, where each parameter $\varphi_{(i)}$ is sampled from a Gaussian distribution defined in terms of $[\mu^i, \sigma^i]$, which allows the agent to influence the applied control action without being directly involved in the control loop. This maintains the stability properties of the low-level controllers.

The learning-based controller must learn how to tune $[\mu^i, \sigma^i]$ for each controller i depending on the environmental conditions. We use therefore domain randomization (DR) to train the learning-based controller. DR considers an extended uncertainty set, including wind and position noise. Given a bounded set of K uncertain parameters/faults $\xi \in \mathbb{R}^K$, DR uniformly samples from ξ at each episode such that the supervisory controller experiences a different disturbance/fault in the environment. DR enables us to generate a set of

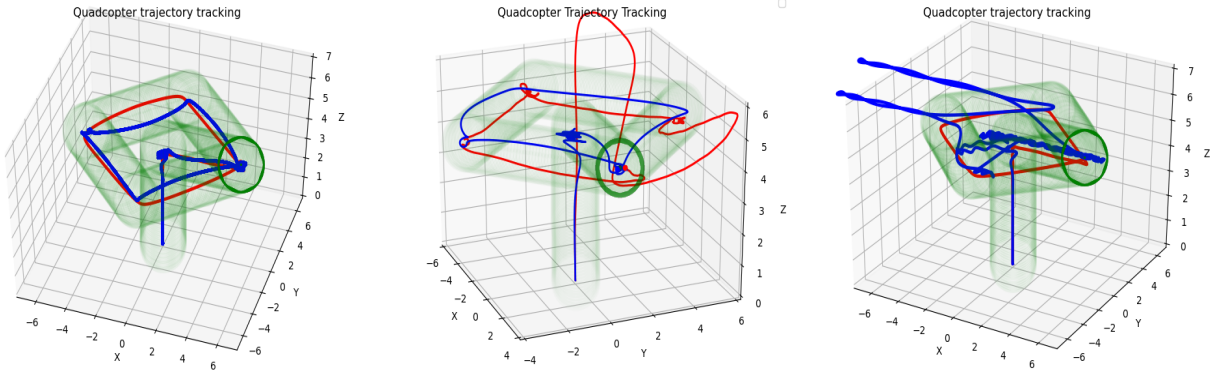


Fig. 2: Trajectories executed under different operating conditions when using **Low-gain controllers (Blue)** or **High-gain controllers (Red)**. The left plot shows nominal performance, the middle plot shows a low-gain controller is more robust to attitude noise, and the right plot highlights the high-gain controller is more robust to rotor faults.

independent MDPs to cover the distribution of $\mathcal{P}$ such that the agent learns to generalize over $\mathcal{P}$. The training process is therefore similar to standard DR but the agent optimizes *across the action space of low-level controllers instead of directly optimizing across the state space of the environment*. In other words, the agent *learns an importance sampling strategy* to accommodate the changes in the environment.

One advantage of our approach is that it allows us to learn/define the low-level and supervisory level controllers separately. This is especially useful when the low-level control problem is well understood, i.e., the controller can be defined analytically and/or is relatively easy to tune for different scenarios. On the other hand, learning how to adapt from one scenario to another is the job of the supervisory controller, which can be learned through RL algorithms such as Proximal Policy Optimization [28]. Moreover, the supervisory controller preserves the stability properties of the individual policies even if it is learned through RL [6]. We demonstrate this in the following section.

C. Stability Properties

This section describes the stability properties of our approach, and the range of environments to which the approach is applicable.

The weighted extension of multiple-model adaptive control (WMMAC) has been well studied, e.g., [29], [6]. We define a control system within this framework using $\Psi = \langle C, \Phi, E \rangle$, where we assume that: (1) we have a collection $\mathcal{C} = \{c_1, \dots, c_m\}$ of controllers, with a distribution (or weight function) over $\mathcal{C}$, denoted $\varphi = \{\varphi_1, \dots, \varphi_m\}$, such that $\sum_j \varphi_j = 1$; (2) each controller c_i , with a parameter set θ_i , is tuned to optimize the performance of a corresponding model $\phi_i, i = 1, \dots, m$ for environment $E_i \in E$, such that ϕ_i is in model set Φ and optimizes the performance in E_i .

Given this framework, a weighted control output is guaranteed to be stabilizing as described below.

Lemma: A WMMC system Ψ is stable if the following hold (see [30]):

- $\phi_j \in \Phi$, for some $j \in 1, \dots, m$, i.e., the true plant model is included in the fixed models of the model set

Φ .

- each model $\phi_i \in \Phi$ can generate a measured signal that is within a fixed bound $B \in \mathbb{R}^n$ of the true signal.
- every controller c_i is locally stabilizing with respect to its model ϕ_i and parameter set θ_i .

Because each weighted control signal is a convex combination of controllers in the control set C , the set of all control combinations forms a convex hull C bounded by C . We can thus define the set of environments for which stable controllers exist under Ψ as follows:

Lemma: Given a WMMC system Ψ defined over controller/model set C/Φ , with corresponding control parameters $\Theta = \{\theta_1, \dots, \theta_m\}$, any environment E with parameters P_E such that the optimal controller has parameters falling within the convex hull of Θ is stabilizable using Ψ .

This second Lemma thus defines the range of environments to which our approach is applicable. We will empirically demonstrate this using a quadcopter with a set of 2 base controllers, for which we show a range of unknown fault environments are stabilizable.

V. QUADCOPTER EXPERIMENTS

The quadcopter trajectory tracking implementation is based on an open-source python simulator [31] and integrated with stable baselines [32] to create a custom quadcopter training environment.¹ The experimental procedure can be broken into four steps. The low-level controllers are tuned for different operating conditions offering contrasting behaviour. The parameter bounds for the faults and disturbances and the reward function to train the agent are defined next. The agent is then trained using domain randomization on the bounded parameter space. Finally, the robustness improvement obtained through training is evaluated by comparing the performance of the low-level controllers on their own, a randomized high-level controller, and the presented hybrid approach after training.

¹A repository containing all simulation files needed to replicate the experiments is made available under "<https://github.com/YvesSohege/ICRA21-QuadcopterExperiments>".

Domain	Level 1	Level 2	Level 3	Level 4
Attitude Noise	0.2 rad	0.4 rad	0.6 rad	0.8 rad
Position Noise	1 m	2 m	3 m	4 m
Wind Gust	3 m/s	6 m/s	9 m/s	12 m/s
Rotor LOE	5%	10%	15%	20%

TABLE I: Quadcopter domain parameters and corresponding disturbance magnitudes for each level.

A. Operating Conditions

We investigate four common disturbances to the quadcopter including rotor loss of effectiveness, wind gusts, position sensor noise, and attitude sensor noise. We denote a multiplicative rotor fault model with parameter $0 \leq \varsigma_i \leq 1$ for $i = 1, \dots, 4$, where $\varsigma_i = 0$ corresponds to nominal function and $\varsigma_i = 1$ to total failure. Wind gusts are implemented using the Dryden turbulence model [33]. Wind direction and the rotor to experience the fault are selected randomly. We model noise using a zero-mean Gaussian distribution of varying magnitudes. Position noise is applied to x, y , and z in meters and attitude noise to ψ and θ in radians. We partition the domain parameter space into *levels* of increasing disturbance magnitudes. The full parameter ranges and for the domains and levels can be found in Table I.

Axis of control	$\mathcal{P}$	$\mathcal{I}$	$\mathcal{D}$
X-Position	300	0.04	450
Y-Position	300	0.04	450
Z-Position	7000	4.5	5000
Roll - C1	24000	0	12000
Pitch - C1	24000	0	12000
Roll - C2	4000	0	1500
Pitch - C2	4000	0	1500
Yaw	1500	1.2	0

TABLE II: PID parameters for low-level model-based control architecture.

B. Trajectory tracking task and reward definition

When a fault or disturbance occurs the quadcopter starts to deviate from its path and can no longer track a reference trajectory. However, in urban scenarios, this could lead to collisions with obstacles. We consider the idea of operational volumes that create a 1-meter **safe zone** around the trajectory which represents the acceptable level of deviation to avoid collisions. The goal of the agent then is to learn how to stay inside the safe zone. The agent receives a large positive reward if the goal location is reached successfully and a small negative reward (-1) for every step outside the safe zone. For an episode with δ steps outside of the safe zone, the reward is defined as $\delta * 2$ if the goal is reached and $-\delta$ in case of failure. This focuses the learning on episodes where the quadcopter deviates outside of the safe zone but is able to re-stabilize to reach the goal. When no deviation occurs the agent is performing well and no rewards are given, which is shown in Figure 3. The operational volume should be tuned empirically depending on the magnitude of the disturbances. This is because the agent is only rewarded, both positively and negatively when the quadcopter leaves the operational

volume. Increasing the safety zone will allow tolerating more aggressive faults.

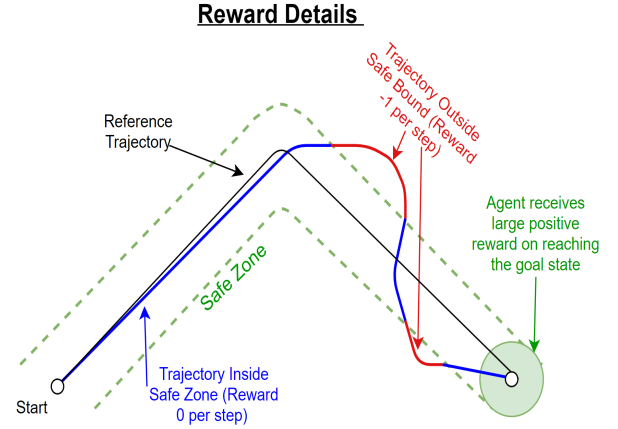


Fig. 3: Shows reference trajectory in black with safe zone in dashed green. When the quadcopter leaves the safe zone the agent is rewarded with a -1 while positive rewards are only possible by reaching the goal.

C. Other Training Setup Details

We define the observation vector as $[x, y, z, \phi, \theta, \psi, X_D, Y_D, Z_D]$, where X_D, Y_D, Z_D represents the goal position. The action space is defined as $[\mu, \sigma]$. We use proximal policy optimization PPO to train a neural network consisting of four layers of 64 neurons. Straight-line trajectories are generated from an initial position on the ground to a random goal location within a 10-meter bounding box. We set a limit of 5000 steps to complete the trajectory. The domain is selected using uniform domain randomization. We train the proposed hybrid architecture for 10 million steps (5770 episodes).

D. Results & Discussion

The performance metric we will use to measure robustness to disturbances is the average number of steps outside of the safe zone. Firstly, we investigate the performance of the high-gain and low-gain PID controllers over the entire fault space which can be seen in Figure 4 **Top-Left** and **Top-Right**, respectively. As discussed previously, the high-gain PID is more robust to rotor faults and the low-gain performs better under attitude noise which is supported by results shown in the heatmaps. Both controllers perform poorly under heavy wind and position noise as they are not tuned for these conditions. Secondly, we are interested in how a uniform randomized blended controller performs across the fault space, seen in Figure 4 **Bottom-Left**. This acts as a baseline to judge how much the supervisory learning component improved the performance by adapting the randomized weighting. We can see that uniform randomized blended control is able to improve the performance under small disturbance magnitudes but is by nature sub-optimal and hence failing on any of the severe disturbances.

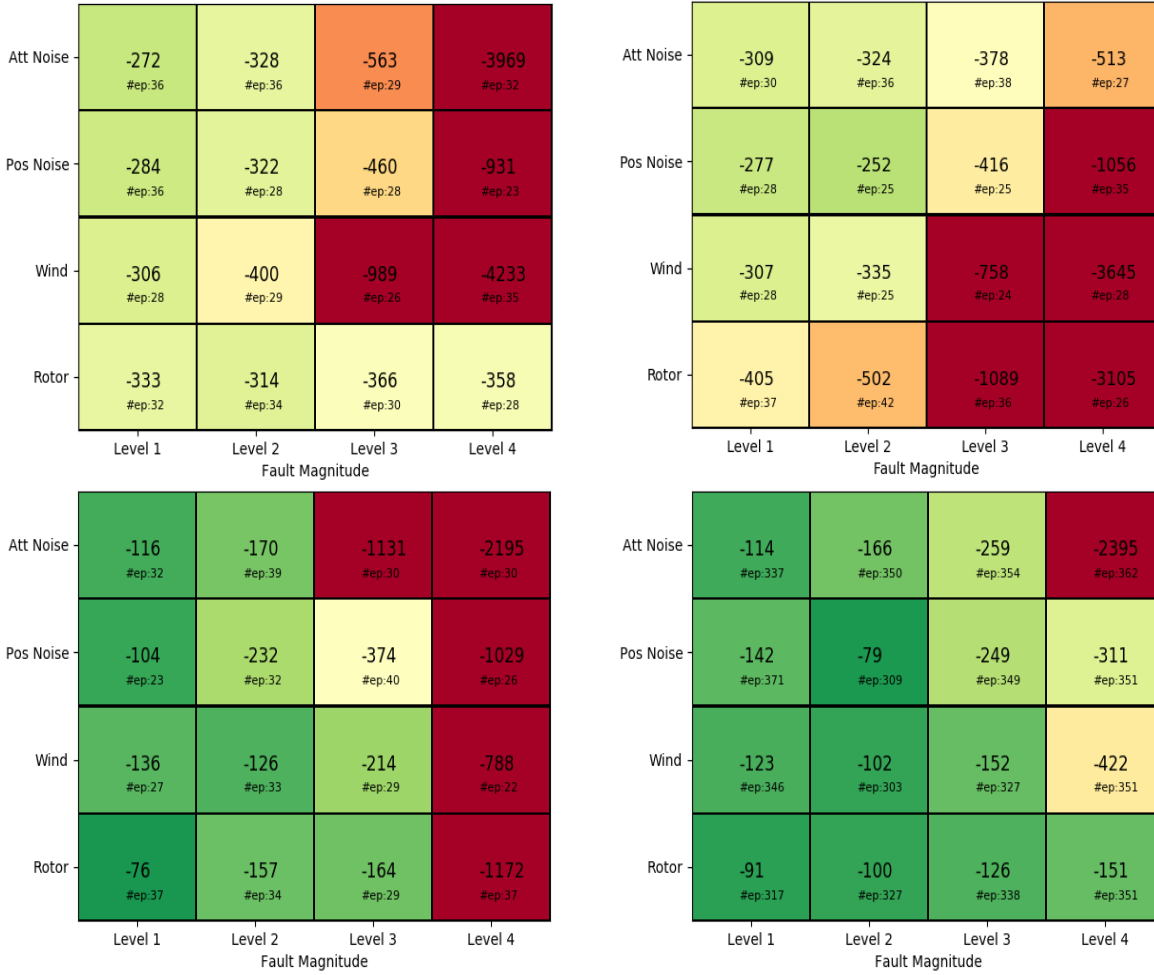


Fig. 4: Performance heatmap showing the average number of steps outside the safe zone under various disturbances. **Top Left - High-gain PID** is robust to rotor faults but suffers on the other domains. **Top Right - Low-gain PID** is robust to attitude noise but suffers on the other domains. **Bottom Left - Uniform Randomized Blended Control** is unable to handle strong disturbances and **Bottom Right - Our Hybrid Approach** significantly outperforms the other approaches over all domains except strong attitude noise.

In contrast, our trained hybrid architecture, Figure 4 **Bottom-Right**, can learn how to adjust the distribution between low-gain and high-gain controller in real-time to improve performance over the entire fault space. A significant improvement can be seen under wind and position noise, for which neither PID controller performs well but the agent is able to learn how to utilize them for this new environment. The only disturbance the agent fails on is severe attitude noise which we attribute to the agent optimizing over all domains. The episode count in each cell shows how many times the agent was exposed to this environment during training.

VI. CONCLUSION

We have described an approach that guarantees key control properties (e.g., stabilizability), in contrast to similar quadcopter control systems [5]. Our hybrid framework enables us to provide stability guarantees based on the model-based controllers while learning a supervisory parameter space that is simpler than the parameter space encompassing the

entire control task. We have validated the FTC approach on urban trajectory-following tasks using a quadcopter. For even relatively extreme rotor faults we show that we can learn to maintain safe trajectories.

Our approach is limited to environments whose corresponding control parameters are contained in the convex hull of the baseline controller parameters. We are currently investigating how we can use RL to learn new baseline controllers for environments not covered by our initial controller set, and initial results indicate that this set of new controllers will grow slowly (and not exponentially) with respect to the complexity of the novel environment sets.

The extension to real-world experiments is expected to perform well as the training is done offline. Standard model parameters such as weight and thrust coefficient can also be randomized during training to overcome the Sim2Real transfer problem. The use of robust controllers should further improve the performance of the presented approach. Further experimental results and full training procedures will be presented in future articles due to space constraints.

REFERENCES

- [1] S. Y. Choi and D. Cha, "Unmanned aerial vehicles using machine learning for autonomous flight; state-of-the-art," *Advanced Robotics*, vol. 33, no. 6, pp. 265–277, 2019.
- [2] N. O. Lambert, D. S. Drew, J. Yaconelli, S. Levine, R. Calandra, and K. S. Pister, "Low-level control of a quadrotor with deep model-based reinforcement learning," *IEEE Robotics and Automation Letters*, vol. 4, no. 4, pp. 4224–4230, 2019.
- [3] W. Koch, R. Mancuso, R. West, and A. Bestavros, "Reinforcement learning for uav attitude control," *ACM Transactions on Cyber-Physical Systems*, vol. 3, no. 2, pp. 1–21, 2019.
- [4] L. Bjarre, "Robust reinforcement learning for quadcopter control," Master's thesis, kth Royal Institute of Technology, School of Electrical Engineering and Computer Science, 2019.
- [5] F. Fei, Z. Tu, D. Xu, and X. Deng, "Learn-to-recover: Retrofitting uavs with reinforcement learning-assisted flight control under cyber-physical attacks," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*, 2020, pp. 7358–7364.
- [6] G. Provan and Y. Sohège, "Fault-tolerant control for unseen faults using randomized methods," in *2019 4th Conference on Control and Fault Tolerant Systems (SysTol)*. IEEE, 2019, pp. 159–164.
- [7] Y. Sohège, M. Quinones-Grueiro, G. Provan, and G. Biswas, "Deep reinforcement learning and randomized blending for control under novel disturbances," in *2020 1st Virtual IFAC World Congress*. IFAC, 2020.
- [8] S. S. Miladi N., Ladhari T., "Enmpc versus pid control strategies applied to a quadcopter," in *New Trends in Robot Control. Studies in Systems, Decision and Control*. Singapore: Springer, 2020, vol. 270, ch. 10, pp. 319–334.
- [9] M. W. Mueller and R. D'Andrea, "Stability and control of a quadcopter despite the complete loss of one, two, or three propellers," in *2014 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2014, pp. 45–52.
- [10] A. Kaba, A. Ermeýdan, and E. Kiyak, "Model derivation, attitude control and kalman filter estimation of a quadcopter," in *2017 4th International Conference on Electrical and Electronic Engineering (ICEEE)*, 2017, pp. 210–214.
- [11] R. C. Leishman, J. C. Macdonald, R. W. Beard, and T. W. McLain, "Quadrotors and accelerometers: State estimation with an improved dynamic model," *IEEE Control Systems Magazine*, vol. 34, no. 1, pp. 28–41, 2014.
- [12] N. S. Özbek, M. Önkol, and M. Ö. Efe, "Feedback control strategies for quadrotor-type aerial robots: a survey," *Transactions of the Institute of Measurement and Control*, vol. 38, no. 5, pp. 529–554, 2016.
- [13] G. Cano Lopes, M. Ferreira, A. da Silva Simões, and E. Luna Colombini, "Intelligent control of a quadrotor with proximal policy optimization reinforcement learning," in *2018 Latin American Robotic Symposium, 2018 Brazilian Symposium on Robotics (SBR) and 2018 Workshop on Robotics in Education (WRE)*, 2018, pp. 503–508.
- [14] J. Hwangbo, I. Sa, R. Siegwart, and M. Hutter, "Control of a quadrotor with reinforcement learning," *IEEE Robotics and Automation Letters*, vol. 2, no. 4, pp. 2096–2103, 2017.
- [15] C. Greatwood and A. G. Richards, "Reinforcement learning and model predictive control for robust embedded quadrotor guidance and control," *Autonomous Robots*, pp. 1–13, 2019.
- [16] M. Blanke, M. Kinnaert, J. Lunze, and M. Staroswiecki, *Diagnosis and Fault-Tolerant Control*. Springer, 2016.
- [17] Y. Chebotar, A. Handa, V. Makovychuk, M. Macklin, J. Issac, N. Ratliff, and D. Fox, "Closing the sim-to-real loop: Adapting simulation randomization with real world experience," in *2019 International Conference on Robotics and Automation (ICRA)*, 2019, pp. 8973–8979.
- [18] K. Rosser, J. Kok, J. Chahl, and J. Bongard, "Sim2real gap is non-monotonic with robot complexity for morphology-in-the-loop flapping wing design," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*, 2020, pp. 7001–7007.
- [19] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, "Sim-to-real transfer of robotic control with dynamics randomization," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*, 2018, pp. 3803–3810.
- [20] M. Efe, "Neural network assisted computationally simple pid control of a quadrotor uav," *IEEE Transactions on Industrial Informatics*, vol. 7, no. 2, pp. 354–361, 2011.
- [21] W. Wiesemann, D. Kuhn, and B. Rustem, "Robust markov decision processes," *Mathematics of Operations Research*, vol. 38, no. 1, pp. 153–183, 2013.
- [22] D. J. Mankowitz, N. Levine, R. Jeong, A. Abdolmaleki, J. T. Springenberg, Y. Shi, J. Kay, T. Mann, T. Hester, and M. Riedmiller, "Robust reinforcement learning for continuous control with model misspecification," in *ICLR*, 2020, pp. 1–28.
- [23] M. A. Abdullah, H. Ren, H. Ou-ammam, V. Milenkovi, R. Luo, M. Zhang, and J. Wang, "Wasserstein Robust Reinforcement Learning," 2019.
- [24] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, "Sim-to-Real Transfer of Robotic Control with Dynamics Randomization," in *IEEE international conference on robotics and automation (ICRA)*. IEEE, 2018, pp. 1–8.
- [25] J. Tremblay, A. Prakash, D. Acuna, M. Brophy, V. Jampani, C. Anil, T. To, E. Cameracci, S. Bochoon, and S. Birchfield, "Training deep networks with synthetic data: Bridging the reality gap by domain randomization," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops*, 2018, pp. 969–977.
- [26] B. Mehta, M. Diaz, F. Golemo, C. J. Pal, and L. Paull, "Active domain randomization," in *Conference on Robot Learning*. PMLR, 2020, pp. 1162–1176.
- [27] S. Wang, N. Anselmo, M. Garrett, R. Remias, M. Trivett, A. Christoffersen, and N. Bezzo, "Fly-crash-recover: A sensor-based reactive framework for online collision recovery of uavs," in *2020 Systems and Information Engineering Design Symposium (SIEDS)*, 2020, pp. 1–6.
- [28] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.
- [29] W. Zhang and Q. Li, "Weighted multiple model adaptive control of time-varying systems," *Journal of Robotics, Networking and Artificial Life*, vol. 1, no. 4, pp. 291–294, 2015.
- [30] W. Zhang and L. Zhao, "Survey and tutorial on multiple model methodologies in modelling, identification and control," *International Journal of Modelling, Identification and Control*, vol. 32, no. 1, pp. 1–9, 2019.
- [31] A. Majumdar, "Python quadcopter simulator," https://github.com/abhijitmajumdar/Quadcopter_simulator, 2018.
- [32] A. Hill, A. Raffin, M. Ernestus, and Gleave, "Stable baselines," <https://github.com/hill-a/stable-baselines>, 2018.
- [33] C. Patel and I. Kroo, "Control law design for improving uav performance using wind turbulence," in *44th AIAA Aerospace Sciences Meeting and Exhibit*, 2006, p. 231.